THADOMAL SHAHANI
TSEC
THADOMAL SHAHANI
TSEC
ENGINEERING COLLEGE

Assignment 9

Aim - To implement a menu driven program to implement a binary search tree. It includes insertion deletion and traversals.

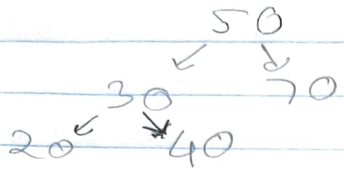
Theory.
A binary search tree is a tree in which the left node has the smaller number and the right node has the larger number than its root.

Insertion in binary search tree involves starting from the root, going right if the value is larger and left if the value is smaller than that of the root. Keep doing this until an empty node is found and insert the number there.

Deletion has ~~cases~~ 3 cases
1. Leaf node - remove the node
2. Node has ~~children~~ one child, replace it with child.
3. Node has two children - replace it with inorder succession.

Tree Traversals are ~~also~~ of 3 kinds
1. Inorder (left root right)
2. ~~Preorder~~ Preorder (Root left Right)
3. Post order (Left Right Root)

Insertion 50, 30, 70, 20, 40



Deletion 30

Traversal

Inorder - 20, 30, 40, 50, 70

Preorder - 50, 30, 20, 40, 70

Post order - 20, 40, 30, 70, 50

Conclusion

Efficiently implement insertion, detion and traversal were performed efficiently. Binary search trees help in maintaining sorted ~~data~~ and allow faster searching.

```
#include <stdio.h>
#include <stdlib.h>

typedef struct node {
   int data;
   struct node *left, *right;
} node;

node* root = NULL;

node* create(int val) {
   node* temp = (node*)malloc(sizeof(node));
   temp->data = val;
   temp->left = temp->right = NULL;
   return temp;
}

node* insert(node* root, int val) {
   if (root == NULL)
      return create(val);
   if (val < root->data)
      root->left = insert(root->left, val);
   else
      root->right = insert(root->right, val);
   return root;
}

node* findMin(node* root) {
   while (root->left != NULL)
      root = root->left;
   return root;
```

```
}

node* deleteNode(node* root, int val) {
   if (root == NULL)
      return NULL;
   if (val < root->data)
      root->left = deleteNode(root->left, val);
   else if (val > root->data)
      root->right = deleteNode(root->right, val);
   else {
      if (root->left == NULL)
         return root->right;
      else if (root->right == NULL)
         return root->left;
      node* temp = findMin(root->right);
      root->data = temp->data;
      root->right = deleteNode(root->right, temp->data);
   }
   return root;
}

void inorder(node* root) {
   if (root != NULL) {
      inorder(root->left);
      printf("%d ", root->data);
      inorder(root->right);
   }
}

void preorder(node* root) {
   if (root != NULL) {
```

```
        printf("%d ", root->data);
        preorder(root->left);
        preorder(root->right);
    }
}

void postorder(node* root) {
    if (root != NULL) {
        postorder(root->left);
        postorder(root->right);
        printf("%d ", root->data);
    }
}

void main() {
    int choice, val;

    while (1) {
        printf("\n--- BST MENU ---\n");
        printf("1. Insert\n2. Delete\n3. Inorder\n4. Preorder\n5. Postorder\n6. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
        switch(choice) {
            case 1:
                printf("Enter value: ");
                scanf("%d", &val);
                root = insert(root, val);
                break;
            case 2:
                printf("Enter value to delete: ");
                scanf("%d", &val);
```

```
        root = deleteNode(root, val);
        break;
    case 3:
        printf("Inorder: ");
        inorder(root);
        break;
    case 4:
        printf("Preorder: ");
        preorder(root);
        break;
    case 5:
        printf("Postorder: ");
        postorder(root);
        break;
    case 6:
        exit(0);
    default:
        printf("Invalid choice!");
  }
}
```

```
--- BST MENU ---
1. Insert
2. Delete
3. Inorder
4. Preorder
5. Postorder
6. Exit
Enter choice: 1
Enter value: 50

--- BST MENU ---
1. Insert
2. Delete
3. Inorder
4. Preorder
5. Postorder
6. Exit
Enter choice: 1
Enter value: 30

--- BST MENU ---
1. Insert
2. Delete
3. Inorder
4. Preorder
5. Postorder
6. Exit
Enter choice: 1
Enter value: 70

--- BST MENU ---
1. Insert
2. Delete
3. Inorder
4. Preorder
5. Postorder
6. Exit
Enter choice: 1
Enter value: 20

--- BST MENU ---
1. Insert
2. Delete
3. Inorder
4. Preorder
5. Postorder
6. Exit
Enter choice: 1
Enter value: 40
```

```
--- BST MENU ---
1. Insert
2. Delete
3. Inorder
4. Preorder
5. Postorder
6. Exit
Enter choice: 3
Inorder: 20 30 40 50 70
--- BST MENU ---
1. Insert
2. Delete
3. Inorder
4. Preorder
5. Postorder
6. Exit
Enter choice: 2
Enter value to delete: 30

--- BST MENU ---
1. Insert
2. Delete
3. Inorder
4. Preorder
5. Postorder
6. Exit
Enter choice: 3
Inorder: 20 40 50 70
--- BST MENU ---
1. Insert
2. Delete
3. Inorder
4. Preorder
5. Postorder
6. Exit
Enter choice: 4
Preorder: 50 40 20 70
--- BST MENU ---
1. Insert
2. Delete
3. Inorder
4. Preorder
5. Postorder
6. Exit
Enter choice: 5
Postorder: 20 40 70 50
```

}